Generalized Forwarding & Software-Defined Networking (SDN)

At a Glance

- Traditional IP forwarding is mainly: **match destination IP → forward out one port**.
- Generalized forwarding is: **match many header fields → execute flexible actions**.
- SDN separates **control plane** (decide rules) from **data plane** (forward packets fast).
- OpenFlow programs packet switches using a **flow table**: **Match + Counters + Actions**.
- With flow tables we can implement routing, switching, firewalling, load balancing, and virtual networks.

1 Why "Generalized" Forwarding?

Traditional Destination-Based Forwarding

In classic routers, forwarding is usually:

Match: Destination IP $\Rightarrow$ **Action: Forward to one output port**

This is fast and simple, but it cannot easily express many real network policies.

Generalized Match–Action Idea

A modern packet switch can make forwarding decisions by matching on fields across multiple layers:

Match: (L2/L3/L4 header fields) $\Rightarrow$

Action: (forward/drop/modify/duplicate/send-to-controller)

So forwarding becomes a **programmable abstraction**.

1.1 Examples of "Actions" (Why It is Powerful)

- **Forwarding**: send to one output port (routing), or many ports (multicast).
- **Load balancing**: send flows across multiple links based on source/port/etc.
- **Rewrite headers**: NAT-like behavior (change addresses/ports/VLAN tags).
- **Drop packets**: firewall behavior (block unwanted traffic).
- **Send to a special box**: e.g., Deep Packet Inspection (DPI), IDS/IPS, monitoring servers.

2 SDN Architecture: Control Plane vs Data Plane

The SDN Philosophy

SDN makes the network easier to manage by separating:

- **Data plane**: packet switches forward packets at line rate using rules.
- **Control plane**: a logically centralized controller computes those rules.

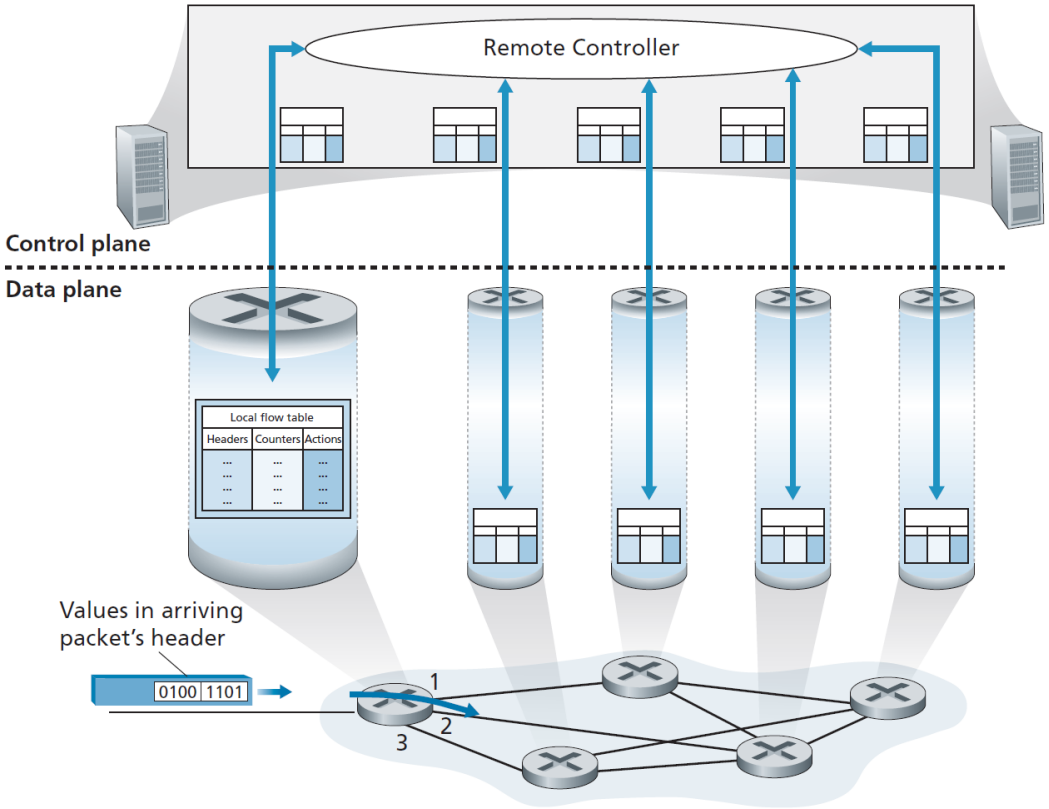

Figure 1: **SDN control plane vs data plane:** remote controller installs/updates flow tables in packet switches.

2.1 What the Figure Means (Teacher Explanation)

- The **remote controller** has a **global view** of the network.
- Each packet switch maintains a local **flow table**.
- When packets arrive, the switch:
  1. matches the packet header against flow table entries,
  2. updates counters,
  3. executes the listed actions.

Why Central Control Helps

With a controller, we can program **network-wide behavior** (routing, security, QoS, load balancing) consistently across many switches.

3 OpenFlow: Flow Tables (Match + Counters + Actions)

Flow Table Entry Structure

Each flow entry in OpenFlow contains:

- **Match**: which packets does this rule apply to?
- **Counters**: statistics updated when rule matches (packets, bytes, last-seen time).
- **Actions**: operations applied to the packet.

3.1 What Happens When a Packet Arrives?

1. Switch checks incoming packet header fields.
2. Finds the **highest-priority matching entry**.
3. Updates counters (e.g., packet count).
4. Performs actions (forward/drop/modify/etc.).

No Match Case

If a packet matches no rule, a switch may:

- drop it, or
- send it to the controller (controller can install a new rule).

4 Match Fields in OpenFlow 1.0 (The Most Important Part)

Matching Across Multiple Layers

OpenFlow 1.0 allows matching fields from:

- **Link layer (L2)**: MAC addresses, Ethernet type, VLAN ID/priority
- **Network layer (L3)**: IP source/destination, protocol, TOS
- **Transport layer (L4)**: TCP/UDP source/destination ports

This allows the network to behave as routing + switching + firewall + NAT + load balancer.

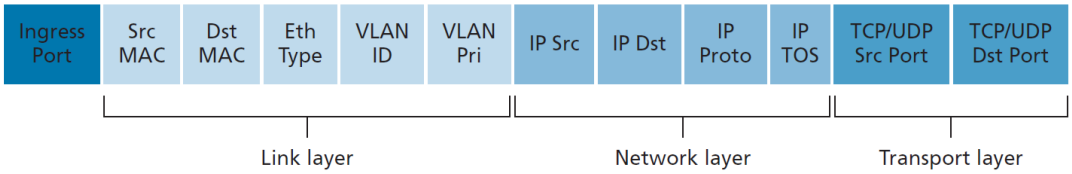

Figure 2: **OpenFlow 1.0 match fields:** packet can be matched using L2, L3, and L4 headers plus ingress port.

4.1 Wildcards and Priority (Very Exam Important)

- A match can include **wildcards**, like:

```
10.3.*.*
```

- Each entry has a **priority**.
- If multiple entries match, the switch selects the **highest priority** rule.

5 Actions in OpenFlow (What the Switch Can Do)

Core Actions

- **Forward**: to a specific port, multiple ports (multicast), or to the controller.
- **Drop**: a rule with no actions means the packet is dropped (firewall).
- **Modify-field**: rewrite header fields before forwarding (NAT-like).

Action Examples

- Forward(out=3) $\Rightarrow$ behaves like routing/switching.
- Drop $\Rightarrow$ behaves like a firewall rule.
- Rewrite(IP dst = X) then Forward(out=2) $\Rightarrow$ can implement NAT / service steering.

6 OpenFlow Example Network (Hosts + Switches + Controller)

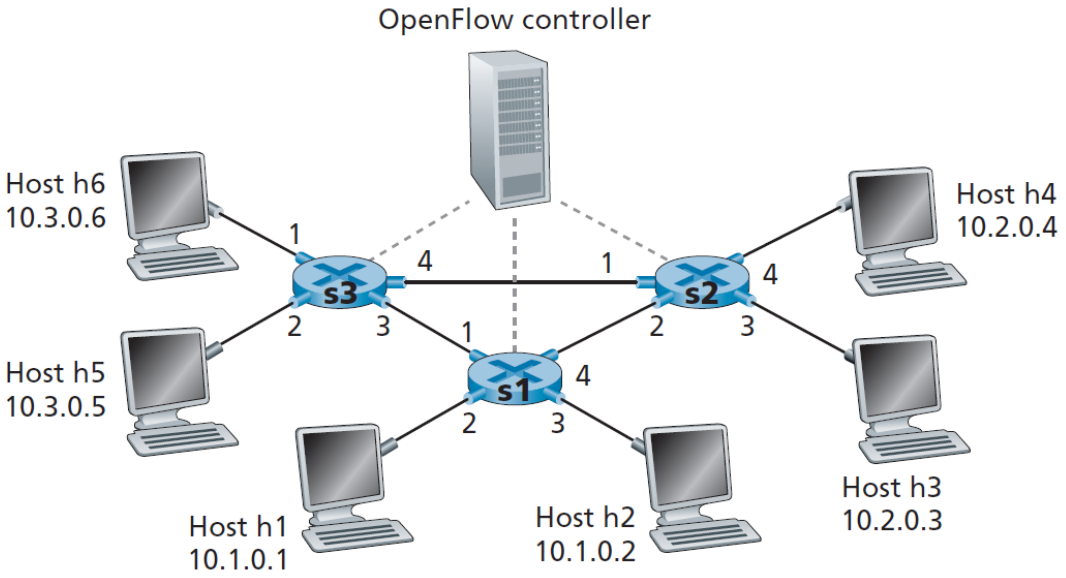

Figure 3: **Example OpenFlow network:** 3 packet switches (s1,s2,s3), 6 hosts, and a controller. Ports are numbered per switch.

6.1 How to Read This Diagram

- Each switch has ports labeled (1,2,3,4).
- Hosts have IPs:
  - h1,h2 in `10.1.*.*`
  - h3,h4 in `10.2.*.*`
  - h5,h6 in `10.3.*.*`
- Controller (dashed lines) manages flow rules in each switch.

7 Three Classic Match–Action Examples (Routing, Load Balancing, Firewall)

7.1 Example 1: Simple Forwarding Policy (Avoid a Link)

Goal

Traffic from `10.3.*.*` (h5/h6) to `10.2.*.*` (h3/h4) should go:

$$s3 \rightarrow s1 \rightarrow s2$$

and should avoid the direct link between s3 and s2.

What Rules Look Like (High Level)

- On s3: match (src=10.3.*.*, dst=10.2.*.*) ⇒ forward toward s1.
- On s1: match (coming from s3, dst=10.2.*.*) ⇒ forward toward s2.
- On s2: match dst=10.2.0.3 ⇒ to h3; match dst=10.2.0.4 ⇒ to h4.

7.2 Example 2: Load Balancing (Split Traffic by Ingress Port)

Goal

Traffic to `10.1.*.*` is split:

- From h3 (port 3 on s2) go directly to s1,
- From h4 (port 4 on s2) go via s3 then to s1.

Key SDN Insight

Destination-only forwarding cannot do this easily, but match–action can, because we can match on **ingress port** (or source IP).

7.3 Example 3: Firewall (Allow Only Specific Sources)

Goal

Only traffic coming from `10.3.*.*` should be allowed to reach h3/h4. All other traffic is blocked by default.

Firewall Behavior in Flow Tables

Firewalling can be done by:

- creating explicit allow rules for `10.3.*.*`,
- not creating allow rules for others (so they get dropped).

8 Beyond OpenFlow: P4 (One Paragraph, Exam Useful)

Why P4 is Mentioned

Flow tables provide limited programmability (match then action). P4 is a more powerful language designed to program packet processing in a protocol-independent way, while still targeting high-speed forwarding hardware.

9 Final Summary (Must Memorize)

Exam-Ready Key Points

- **Generalized forwarding** = match many header fields + flexible actions.
- **SDN** separates **control plane** (controller) and **data plane** (switches).
- **OpenFlow** uses a **flow table**: Match + Counters + Actions.
- Matching can use L2/L3/L4 fields + ingress port; rules support wildcards and priorities.
- Actions include forward, drop, modify-field, mirror, and send-to-controller.
- Flow tables can implement routing, switching, firewalling, load balancing, and virtual networks.

SDN makes networks programmable. Generalized forwarding is the core mechanism.